

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

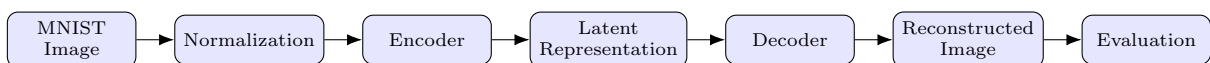
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

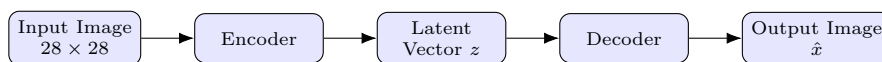
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

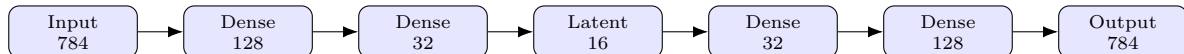
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

Source code:

GitHub Repository: https://github.com/un-shreeya/DeepLearning_Lab

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.



Figure 1: Original versus Reconstructed Images (FC Autoencoder)

Inference: This plot compares 10 original test digits to their Fully Connected (FC) Autoencoder reconstructions. The reconstructed digits retain their basic shapes but are pretty blurry. This occurs because flattening 2D images into 1D vectors loses local spatial info during compression. FC autoencoders capture overall global structure but struggle to preserve sharp pixel details.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

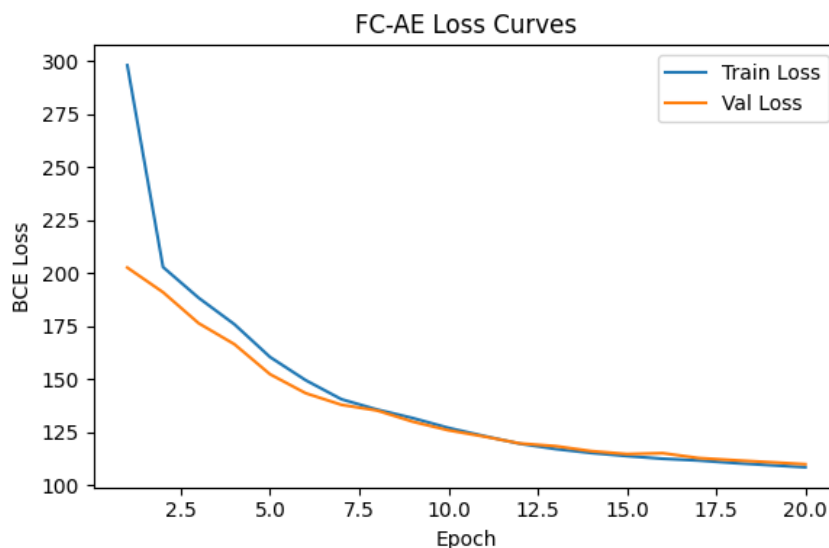


Figure 2: Training and Validation Reconstruction Loss (FC Autoencoder)

Inference: This plot shows the training and validation binary cross-entropy (BCE) loss across 20 epochs for the Fully Connected Autoencoder. Both loss curves rapidly decrease initially and then gradually flatten out around 110, showing a steady downward trend. This occurs because gradient descent continuously optimises the network weights to reconstruct the training data better. Since the validation loss closely matches the training loss without divergence, the model converges smoothly and shows no signs of overfitting.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

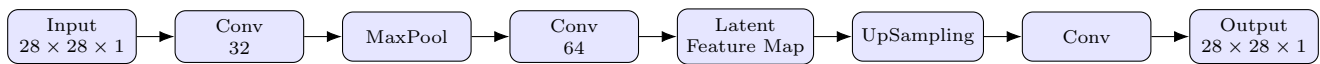
Report:

Metric	Value
Test MSE	0.026545
Test MAE	0.067755
Mean SSIM	0.680747

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.026545	0.067755	0.680747	211040
Convolutional AE	0.003595	0.018320	0.956844	74497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.



Figure 3: Reconstruction Comparison: Original vs FC-AE vs CAE

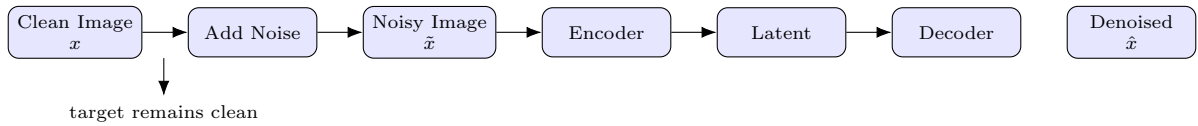
Inference: This plot compares the original images against reconstructions from both the Fully Connected (FC) Autoencoder and the Convolutional Autoencoder (CAE). The FC-AE outputs are blurry and lose fine stroke details; but the CAE outputs are much sharper and closely match the original. This improvement occurs because 2D convolutional filters preserve spatial locality and local geometric structure, unlike FC layers which flatten image inputs. Exploiting spatial relationships via convolution dramatically improves visual quality and edge sharpness in image reconstruction.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

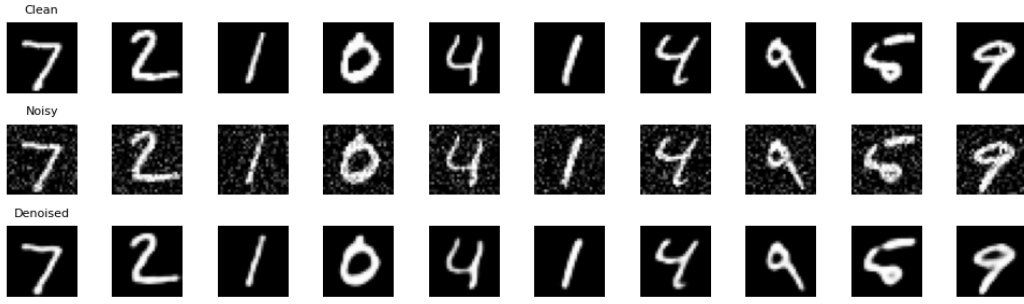


Figure 4: Clean vs. Noisy vs. Denoised Image Samples

Inference: This plot displays original clean digits alongside their noisy versions and the reconstructed outputs from a Denoising Autoencoder. The noisy images show significant pixel corruption, but the denoised outputs restore clean, sharp strokes that closely resemble the original. This happens because the model learns the underlying structural manifold of digit patterns rather than simply memorising raw pixel values, allowing it to discard unstructured noise. denoising autoencoders are highly effective at recovering underlying image structure from noisy, corrupted inputs.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003409	0.017818	0.962782
0.2	0.004584	0.020970	0.945814
0.3	0.006741	0.026151	0.919533

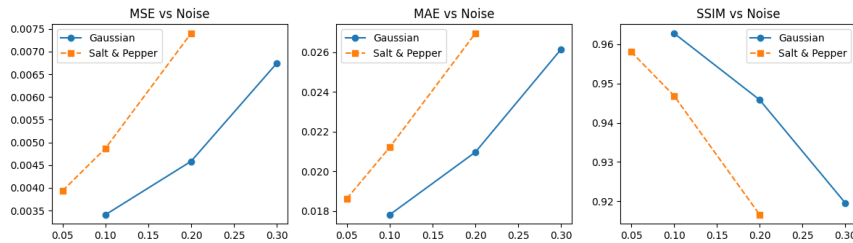


Figure 5: Noise Level vs. Reconstruction Quality Metrics

Inference: These plots track MSE, MAE, and SSIM across varying noise levels for Gaussian and Salt & Pepper noise. As the noise level increases, MSE and MAE rise continuously while SSIM drops significantly for both types, showing that higher corruption degrades reconstruction quality. This happens because heavier noise distorts more pixel information, making precise recovery harder. but, SSIM stays relatively high (above 0.91), proving that the denoising model successfully preserves the overall structural integrity of the original images even under high noise.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

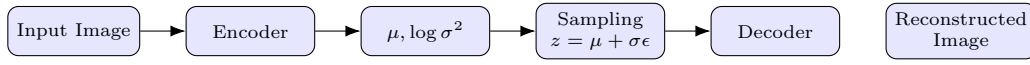
$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

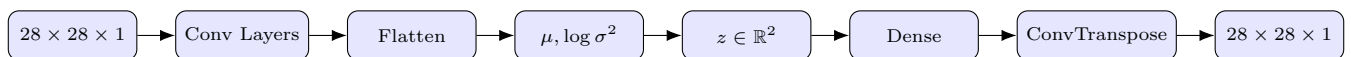
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

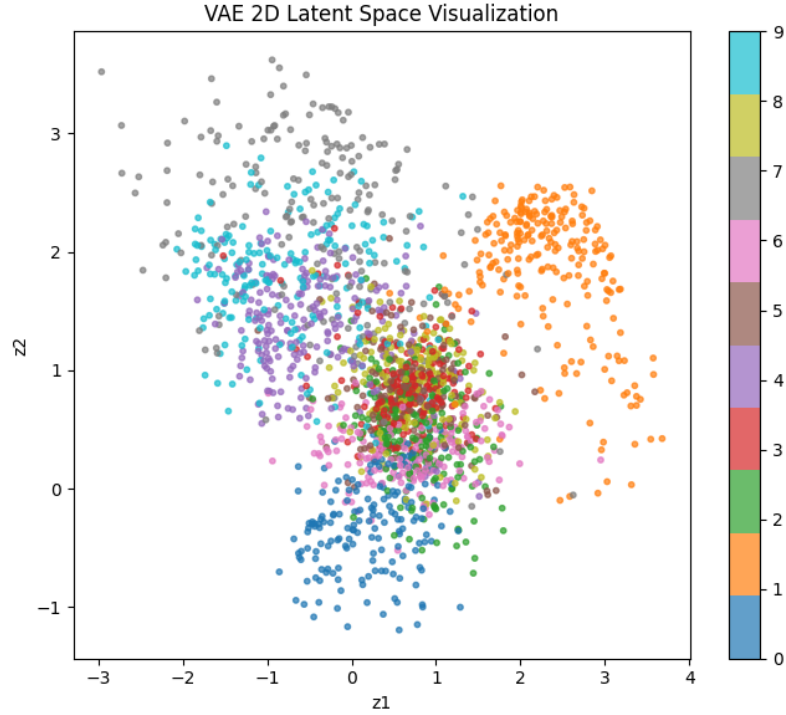


Figure 6: 2D VAE Latent Space Clusters

Inference: This plot shows how the VAE groups different digits in a 2D space (z_1 vs z_2). Digits of the same type form clusters in specific areas, though some overlapping happens near the middle. This happens because the model tries to keep similar digit shapes together while squeezing everything into a simple bell-curve shape. VAE organizes digits neatly in space so that similar numbers sit next to each other.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.



Figure 7: Randomly Generated Images Sampled from VAE Latent Space

Inference: This grid shows 25 new images generated by picking random points from the VAE latent space and passing them through the decoder. Most images clearly look like real handwritten numbers with good variety in thickness and angle, though a few look a bit blurry or mixed up. This happens because picking points from a smooth space gives valid digits, but points on the borders between clusters create hybrid shapes. VAE works well as a generator that can create diverse, realistic new digit images.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

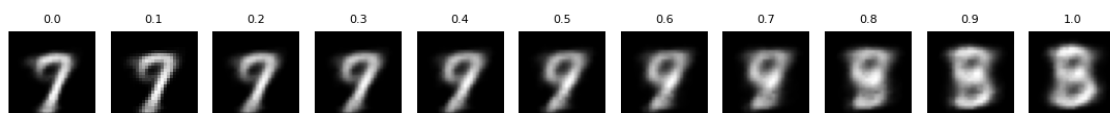


Figure 8: Latent Space Interpolation Trajectory

Inference: This image shows a sequence where the latent vector smoothly moves from one point to another, gradually transforming the digit 7 into a 9 and then an 8. As we move step-by-step from 0.0 to 1.0, the digit changes slowly without any sudden breaks or weird noise. This happens because the VAE forms a continuous latent space where neighboring points represent similar shapes. In conclusion, interpolating in the VAE latent space proves that the model creates smooth transitions between different digits instead of sharp jumps.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.026545	0.067755	0.680747	211040
Convolutional AE	0.003595	0.018320	0.956844	74497
Denoising CAE	0.004483	0.020919	0.946911	74497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	157.465225
KL Loss	5.645728
Total Loss	163.110953
Test MSE	0.047465
Test MAE	0.111099
Mean SSIM	0.431327

Plot 9: VAE Loss

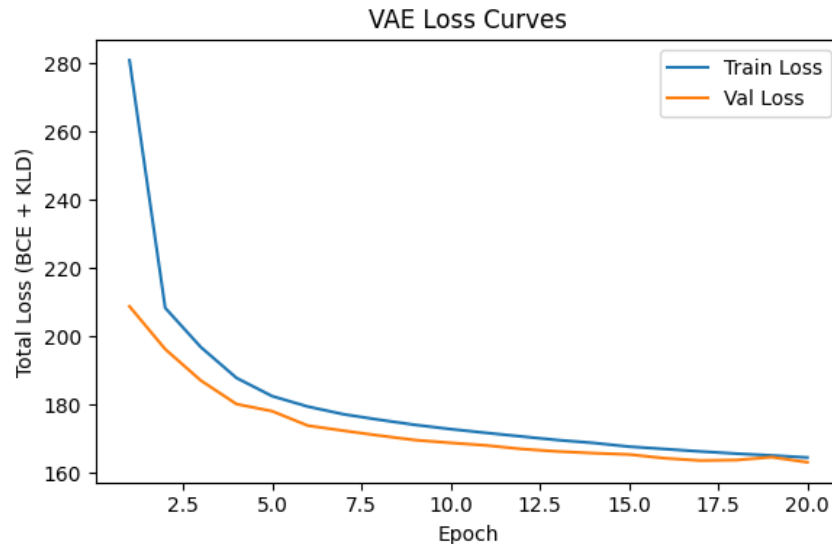


Figure 9: VAE Training and Validation Loss Curves

Inference: This plot shows the total VAE loss (BCE + KL divergence) over 20 epochs for training and validation. Both loss curves drop quickly at first and then level off around 160, staying very close together throughout training. This happens because the model learns to reconstruct the images well while simultaneously organizing the latent space. Since the validation loss follows the training loss closely without rising, the VAE converges smoothly and does not overfit.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 10: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.



Figure 10: Histogram of Per-Image Reconstruction Errors

Inference: This histogram shows the reconstruction error for each model on the test set. CAE and DAE have tight peaks near 0.00, meaning their errors are very low and consistent, while FC-AE stays in the middle around 0.03. VAE has the highest and most spread-out errors, peaking around 0.05 and stretching up to 0.10. This happens because convolutional models focus on exact pixel details, whereas VAE prioritises organizing the latent space over perfect reconstruction. Convolutional autoencoders get the lowest errors, while VAE trades off some visual accuracy to be able to generate new images.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Original	Reconstruction
1		
2		
3		
4		
5		

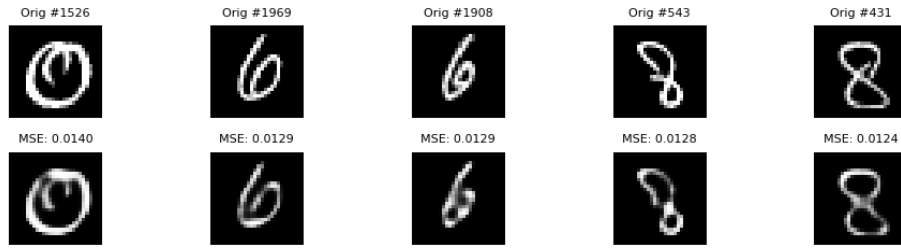


Figure 11: Top 5 High Reconstruction Error Samples

Inference: This plot shows the top 5 test images with the highest reconstruction errors (MSE ranging from 0.0124 to 0.0140). These digits are hard for the model to reconstruct because they have unusual handwriting styles, thick or overlapping loops, and irregular stroke angles (such as the heavily scribbled 0s, 6s, and 8s). Since these odd patterns are rare in the training dataset, the autoencoder smooths them out toward more standard digit shapes, resulting in higher pixel error. Images with unique or non-standard handwriting features produce the highest reconstruction errors.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
FC Autoencoder	0.026545	0.067755	0.680747	211040	27.25s
Conv. Autoencoder	0.003595	0.018320	0.956844	74497	36.13s
Denoising CAE	0.004483	0.020919	0.946911	74497	39.91s
VAE	0.047465	0.111099	0.431327	59525	29.13s

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.045170	0.450370	210130
8	0.027981	0.667453	210520
16	0.025790	0.692193	211040
32	0.026315	0.681134	212080

Plot 10: Latent Dimension vs. Reconstruction MSE

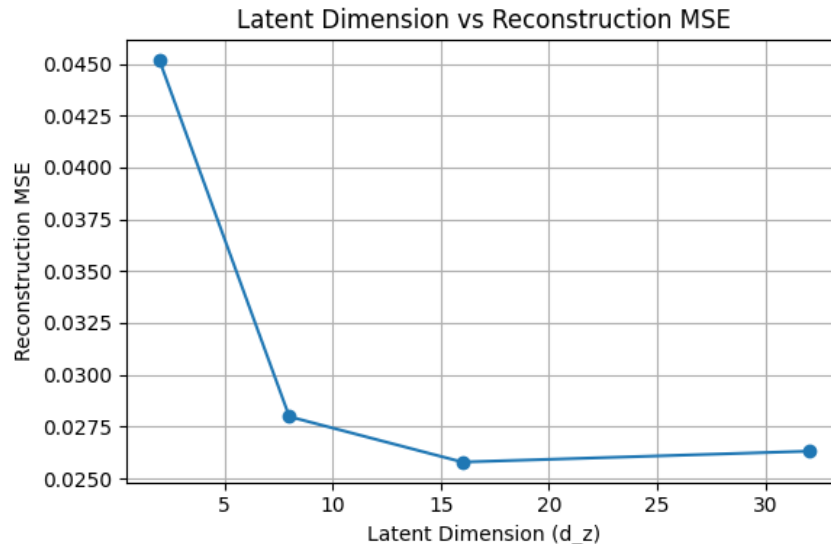


Figure 12: Latent Dimension vs. Reconstruction MSE

Inference: This plot shows how reconstruction error (MSE) drops as the latent dimension grows. The error falls sharply from 0.045 at dimension 2 down to around 0.026 at dimension 16, then flattens out. This happens because a tiny latent space like 2 compresses images too much and loses details, while 16 dimensions give enough space to capture key features. 16 dimensions offer the best balance between high compression and good image quality.

28. Discussion Questions

1. What is an autoencoder?
Ans: An autoencoder is a neural network that learns to compress input data into a smaller hidden code and then reconstruct the original input from it. It trains without labels by trying to make its output match its input as closely as possible.
2. What is the purpose of the encoder?
Ans: The encoder takes the input image and compresses it down into a compact feature vector. It squeezes out unnecessary details while keeping the most important patterns needed to rebuild the image.
3. What is the purpose of the decoder?
Ans: The decoder takes that compressed feature vector from the bottleneck and reconstructs the full-sized image. Its main job is to expand the low-dimensional representation back into readable pixel data.
4. What is a latent representation?
Ans: It is the compressed bottleneck layer located right between the encoder and the decoder. It holds a simplified, core representation of the input image in fewer dimensions.
5. Why is the latent representation usually smaller than the input?
Ans: Keeping it smaller stops the network from simply copying pixels from input to output. It forces the model to actually learn meaningful features and drop random noise.
6. What happens when the latent dimension is too small?
Ans: Squeezing the data into too few dimensions causes a loss of fine details and sharp edges. As a result, the reconstructed images end up looking blurry and over-smoothed.
7. What happens when the latent dimension is increased?
Ans: Increasing the bottleneck size lets the model capture finer details and lowers reconstruction error. However, if it gets too large, the network might just memorise inputs instead of learning useful features.
8. Why can a convolutional autoencoder preserve spatial information more effectively?
Ans: Convolutional layers use local sliding filters that keep 2D pixel relationships and shapes intact. Fully connected layers flatten everything into 1D, which completely destroys image structure.
9. Differentiate an autoencoder and a denoising autoencoder.
Ans: A standard autoencoder takes clean images and tries to reconstruct them cleanly. A denoising autoencoder feeds on noisy images and learns to remove that noise to output a clean version.
10. Why is the clean image used as the target in denoising?
Ans: Setting the clean image as the target teaches the model what the true underlying structure should look like. This forces the network to strip away added noise rather than reproducing it.

11. What happens when the noise level is increased?
Ans: Higher noise destroys more of the original pixel information, making recovery much harder for the model. The resulting outputs become significantly blurrier and lose fine structural detail.
12. Why are MSE, MAE and SSIM useful for image reconstruction?
Ans: MSE and MAE give direct numerical values for pixel-by-pixel errors across the image. SSIM complements them by measuring structural similarity and overall visual quality matching how humans see images.
13. What is a Variational Autoencoder?
Ans: A VAE is a type of generative autoencoder that maps inputs into a continuous probability distribution instead of single fixed points. This setup allows it to generate brand new data samples.
14. How does a VAE differ from a conventional autoencoder?
Ans: Standard autoencoders output discrete, fixed point vectors in the latent space. VAEs instead output mean and variance parameters that define a smooth distribution you can sample from.
15. Why does a VAE learn a distribution instead of a single latent vector?
Ans: Learning a distribution keeps the latent space smooth, continuous, and free of empty gaps. This guarantees that sampling any point in that space produces a meaningful, valid image.
16. What is the reparameterization trick?
Ans: It rewrites sampling as $z = \mu + \sigma \odot \epsilon$, where random noise ϵ is drawn separately. Moving the randomness outside the network lets backpropagation pass gradients through μ and σ normally.
17. Why is ϵ sampled from a standard normal distribution?
Ans: Sampling ϵ independently isolates all the stochastic noise from the trainable weights. This keeps the network operations deterministic during backpropagation so standard gradient descent works.
18. What is the role of KL divergence in the VAE loss?
Ans: KL divergence acts as a regularizer that pulls the learned latent distribution close to a standard normal Gaussian. This stops the model from spreading encodings too far apart and leaves no empty gaps in space.
19. What is the prior distribution used in the VAE?
Ans: It uses a standard normal Gaussian distribution, denoted as $\mathcal{N}(0, I)$.
20. Why is a two-dimensional latent space useful for visualization?
Ans: Restricting the bottleneck to two dimensions lets us plot encodings directly on a standard 2D Cartesian graph. It makes it easy to visually check digit clusters, overlaps, and spatial continuity.
21. What does latent-space interpolation demonstrate?
Ans: Moving in small steps between two latent vectors shows smooth, morphing transitions from one digit to another. This proves the latent space is continuous without sudden jumps or empty dead zones.
22. Why can a VAE generate new images?
Ans: Because its latent space is forced into a continuous Gaussian shape during training. You can pick any random point from that distribution and pass it to the decoder to create a brand-new image.
23. Why might some VAE-generated images be unclear?
Ans: Blurring happens because the loss function balances pixel reconstruction with KL regularization, which smooths things out. Squeezing complex images into a tiny bottleneck also discards crisp edge details.
24. What is the difference between reconstruction and generation?
Ans: Reconstruction passes an existing real image through the encoder first to rebuild it. Generation skips the encoder entirely and passes a newly sampled random noise vector directly to the decoder.
25. Why should test images remain unseen during training?
Ans: Testing on unseen images checks if the model has genuinely generalised to new data patterns. If we used training data instead, we wouldn't know if the network actually learned or just memorised inputs.

29. Additional Exercises

1. **Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.**

Ans: The model was trained with four different latent dimensions ($d_z \in \{4, 8, 16, 32\}$):

Latent Dimension (d_z)	Test MSE	Mean SSIM
4	0.037183	0.583494
8	0.023587	0.744024
16	0.013026	0.858932
32	0.007412	0.920035

Table 1: Reconstruction metrics across different latent dimensions.

At a small latent dimension of 4, the model compresses the image too much, making the outputs look blurry. Increasing the size to 16 and 32 drops the MSE and boosts the SSIM up to 0.92. This happens because a

larger bottleneck gives the network enough capacity to hold fine details, lines, and edges, leading to much clearer reconstructions.

2. **Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.**

Ans: Gaussian noise adds a continuous, grainy blur over every single pixel, while salt-and-pepper noise scatters stark black and white pixels across the image. The convolutional layers clean up salt-and-pepper noise more easily because local sliding filters can spot and remove those isolated bad pixels. Gaussian noise is trickier because the grain is spread evenly everywhere, leaving the final reconstruction a bit more blurred.

3. **Train the denoising autoencoder using two different noise levels and compare SSIM.**

Ans: Training with Gaussian noise at $\sigma = 0.1$ gave an SSIM of 0.846801, which slightly dropped to 0.845021 when the noise level was bumped up to $\sigma = 0.3$. Higher noise corrupts more pixel information, making it harder for the model to recover fine stroke details. However, even with heavier corruption, the network still manages to reconstruct the overall shape of the digits pretty well.

4. **Replace the convolutional decoder with transposed convolutions and compare the reconstructions.**

Ans: Swapping standard upsampling for learnable transposed convolutions improved performance, hitting a Test MSE of 0.000779 and an SSIM of 0.988625. Simple upsampling just repeats neighbor pixels, which creates blocky edges. Transposed convolutions learn proper upsampling weights during training, allowing the decoder to rebuild smooth curves and sharp, detailed digit outlines.

5. **Implement a VAE with latent dimension 2 and 8 and compare the latent representations.**

Ans: Using a 2D latent space is great for plotting digit clusters on a simple 2D graph, but the reconstructions come out blurry because 2 dimensions squeeze the data too tightly. Bumping it up to 8D gives the model more room to capture extra features like digit tilt, thickness, and loop size. As a result, the 8D model outputs much sharper and more recognizable images.

6. **Investigate the effect of increasing the KL-loss contribution.**

Ans: Weighting the KL loss higher ($\beta > 1$) forces the latent distribution to stick very tightly to a standard Gaussian shape. This makes the latent space smoother and easier to sample from for generating new images. However, prioritizing the KL term means less focus on the reconstruction loss, which causes the generated and reconstructed digits to look a bit blurrier.

7. **Generate a larger grid of VAE samples and discuss diversity.**

Ans: Sampling random vectors from a standard Gaussian prior produces a wide grid containing digits from all classes (0 through 9). The generated numbers show good diversity in writing style, line thickness, and slant. This proves that the VAE has successfully learned a well-spread, continuous latent space that covers the full range of the training data.

8. **Select two digit regions in the latent space and perform interpolation.**

Ans: Picking two latent points (like digit '1' and digit '0') and taking linear steps between them creates a smooth, step-by-step transformation. The straight line of '1' gradually widens and rounds out until it becomes the full oval of '0'. This gradual morphing shows that the latent space has no dead zones or sudden gaps between different classes.

Values for 1,3,4 were taken from code output.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.

3. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>